

Testování

Karel Richta

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2024

Softwarové inženýrství, SWI
03/2024, Lekce 11

<https://moodle.vspj.cz/course/view.php?id=202893>



Testování, validace, verifikace

- **testování:**

$x \in \text{VstupníData}$.

$\text{JsouSprávně?}(\text{VýstupníData})$

- **validace:** *Vyrobili jsme správný SW - tj. odpovídá požadavkům?*

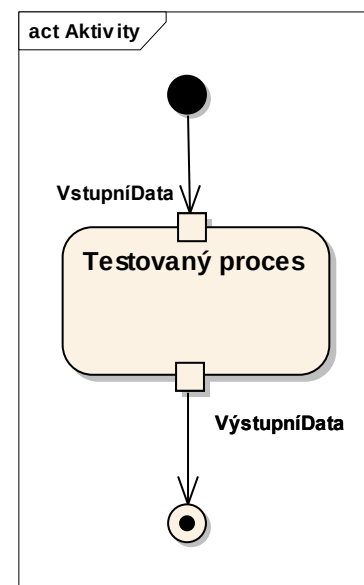
$\forall x \in X \subset \text{VstupníData}$.

$\text{JsouSprávně?}(\text{VýstupníData})$

- **verifikace:** *Vyrobili jsme SW správně - tj. odpovídá specifikaci?*

$\forall x \in \text{VstupníData}$.

$\text{JsouSprávně?}(\text{VýstupníData})$



Příklad

- Uvažujme posloupnosti prvků z X ($X \neq \emptyset$):

kde X je *poset* uspořádaný relací $\leq : X \times X \rightarrow \text{Bool}$

a posloupnosti prvků z X tvoříme operacemi:

$$\underline{\varepsilon : \rightarrow X^*}$$

prázdná posloupnost

$$\underline{\text{cons} : X \times X^* \rightarrow X^*}$$

přidání prvku do čela posloupnosti

- Specifikace funkce řazení **sort**:

$$\underline{\text{sort} : X^* \rightarrow X^*}$$

$$\forall x \in X. \text{sorted}(\text{sort}(x))$$

sorted je požadovaná vlastnost funkce **sort**
(specifikace)

$$\text{sorted}(\varepsilon) = \text{true}$$

$$\text{sorted}(\text{cons}(x, s)) = \text{is-min}(x, s) \wedge \text{sorted}(s)$$

$$\text{is-min}(x, \varepsilon) = \text{true}$$

$$\text{is-min}(x, \text{cons}(y, s)) = x \leq y \wedge \text{is-min}(x, s)$$

Příklad (pokr.)

- Jiná specifikace seřazené posloupnosti:

sorted: $X^* \rightarrow \text{Bool}$

$_ * _ : X^* \times X^* \rightarrow X^*$

spojení posloupností (zřetězení)

$\epsilon * x = x$

spojení s prázdnou posloupností

$x * \epsilon = x$

dtto

cons(x,s) * y = **cons**(x, s * y)

asociativita

sorted(s) =

(length(s) ≤ 1) or

($\forall x,y \in X . (s == s1 * x * s2 * y * s3) \text{ and } (x \leq y)$)

pokud se x a y nacházejí v seřazené
posloupnosti za sebou musí platit $x \leq y$

Příklad (pokr.)

- **Testy:**

sorted(sort(ε)) = true ?

sorted(sort({5, 3, 2})) = true ?

sorted(sort({1, 2, 3})) = true ?

...

- **Jedna implementace:**

sort₁ = $\lambda x:X^*. \varepsilon$ nebo jinak: **sort₁(x) = ε**

sorted(sort₁(ε)) = sorted(ε) = true

sorted(sort₁({5, 3, 2})) = sorted(ε) = true

sorted(sort₁({1, 2, 3})) = sorted(ε) = true

Příklad (pokr.)

- **Validate:**

`sorted(sort( $\epsilon$ )) = true ?`

`sorted(sort({1})) = true ?`

`sorted(sort({3, 2, 1})) = true ?`

...

- **Verifikace:**

$\forall s \in X^*. \text{sorted}(\text{sort}(s))$

indukcí podle délky s :

`sorted(sort( $\epsilon$ ))`

`sorted(sort( $s$ ))  $\Rightarrow$  sorted(sort(cons( $x$ ,  $s$ )))`

Příklad (pokr.)

- Je **sort₁** korektní?

Vzhledem ke specifikaci - **ano**:

$$\forall s \in X^*. \text{sorted}(\text{sort}_1(s)) = \forall s \in X^*. \text{sorted}(\varepsilon) = \text{true}$$

Ale lepší je specifikace:

$$\forall s \in X^*. \text{sorted}(\text{sort}(s)) \wedge \text{is-permutation}(s, \text{sort}(s))$$

Příklad (pokr.)

- A jedna možná implementace řazení **isort**:

isort: $X^* \rightarrow X^*$

isort(ε) = ε

isort(**cons**(x, s)) = **insert**($x, \text{isort}(s)$)

insert: $X \times X^* \rightarrow X^*$

insert(x, ε) = **cons**(x, ε)

insert($x, \text{cons}(y, s)$) = **cons**($y, \text{insert}(x, s)$) if $y < x$

insert($x, \text{cons}(y, s)$) = **cons**($x, \text{cons}(y, s)$) otherwise

- **isort** je korektní vzhledem ke specifikaci (tento fakt může být verifikován indukcí podle délky posloupnosti s).
- Verifikace vyžaduje formalizaci.

Testování

- Testování je proces, kdy se snažíme ověřit kvalitu produktu a případně identifikovat defekty a problémy, které by kvalitu mohly zhoršit.
- Testování softwaru zahrnuje dynamické ověřování správného chování programu na konečné sadě testovacích případů, vybraných z potenciálně nekonečné množiny možností.
 - Dynamický charakter vyjadřuje, že testování vyžaduje provádění programu s určitými vstupními daty.
 - Výběr testovacích případů je nutný a důležitý, neboť ani jednoduchý program nelze vyzkoušet pro všechny případy.
 - Správné chování může být určeno očekávanými výstupními daty (validace), nebo specifikací (verifikace).

Je formalizace smysluplná?

- Pro následující výraz neznáme sice výsledek, ale umíme ho spočítat jednoduchým algoritmem:

$165987 * 2846$

- Umíme také spočítat výsledek výrazu?

$\text{CMIIIX} * \text{MDIV}$

- Jedno možné řešení:

$\text{Arabic2Roman}(\text{Roman2Arabic}(\text{CMIIIX}) * \text{Roman2Arabic}(\text{MDIV}))$

Proč má testování takový význam?

- 40% času vývoje rozsáhlých aplikací představuje jejich ověřování
- až 50% nákladů na vývoj připadá na testování
- 75% všech aplikací má problémy s kvalitou
- pouze 1% aplikací je dokončeno včas, v rámci stanoveného rozpočtu a v požadované kvalitě
- velmi značné riziko, že můj programový systém „spadne do uvedených kategorií“
- nutnost riziko řídit a snížit $\Rightarrow$ nutnost testovat

Co by mělo být cílem testování?

- Ověření:
 - nepredikovatelných následků událostí a stavů (zejména u událostmi řízených programových systémů),
 - chování v reálných podmínkách nasazení,
 - ověření přenositelnosti instalace,
 - ověření chování při havárii a následném zotavení,
 - zákaznické.
- Zajištění kvality programového systému.
- Zajištění důvěry v kvalitu.

Jaké typy testů připadají v úvahu?

- testy základních modulů
- testy uživatelského rozhraní
- integrační testy
- finální testy (funkčnost, zátěž, dokumentace)
- akceptační testy
- profylaxe = pravidelná ověřování ve fázi údržby
- ověření změn (původní kvalita + nové vlastnosti)
- ověření rozšiřitelnosti

Testování

- Testování podle struktury dat
- Testování podle struktury programu
- Testování časových závislostí (u paralelních systémů)

Testování podle struktury dat

Vstup

- EOF
- příliš-dlouhé-slovo
- ...

Výstup

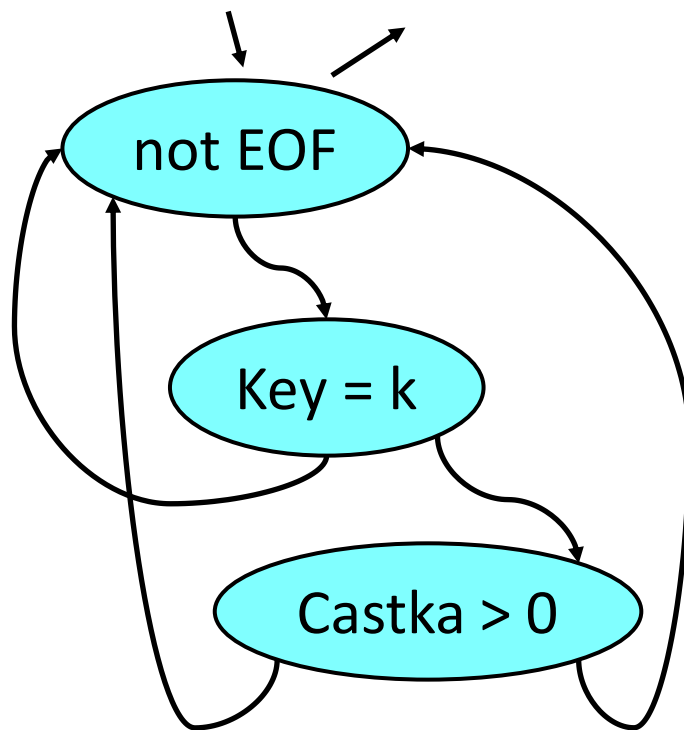
EOF
hlášení chyby

Testování podle struktury programu

```
type Veta = record Key: Klic; Castka: integer end;
function f(baze: file of Veta; k: Klic): integer;
var S: integer; t:Veta;
begin
    reset(baze); S := 0;
    while not eof(baze) do
    begin read(baze,t);
        if (t.Key = k) then
            if (t.Castka > 0) then S := S + t.Castka;
        end;
    f := S
    end;
```


Testování podle struktury programu

(cyklomatické číslo = 3)



1. soubor s jednou větou s jiným klíčem ($\neq k$)
2. soubor s jednou větou se stejným klíčem a zápornou částkou
3. soubor s jednou větou se stejným klíčem a kladnou částkou

Požadavky na proces testování

- Testy jsou založeny na specifikaci a uživatelské dokumentaci.
- Musí být ověřeny minimálně všechny požadavky dané specifikací, popř. závaznými standardy pro věcnou oblast.
- Testy musí být opakovatelné, přesně definované a dokumentované.
- Musí být stanoveno a dodrženo testovací prostředí a testovací podmínky, které musí splňovat podmínku neovlivnění průběhu testu činností jiného programového systému, nebo speciálním hardware.

Příklad specifikace prostředí

Benchmarkové testy TPC (<http://www.tpc.org/>):

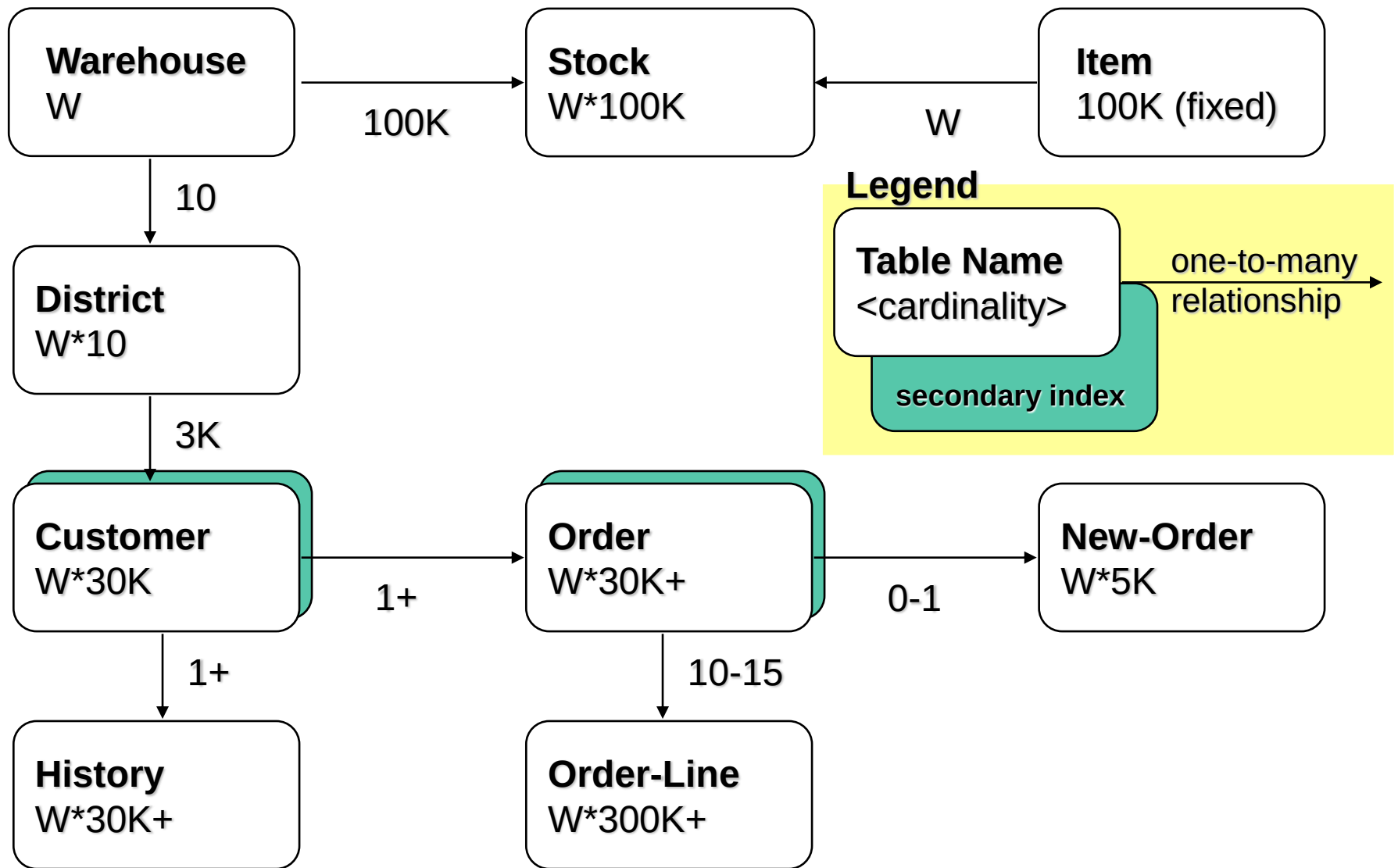
- TPC-A
 - měří výkonnost v aktualizacně náročném DB prostředí typicky OLTP aplikací
- TPC-B
 - hodnocení výkonnosti jádra DB systému s operačním systémem, na kterém DB server běží
- TPC-C
 - modelování komplexnějšího systému
- TPC-D
 - výkonnost DB systémů při dotazech pro podporu rozhodování

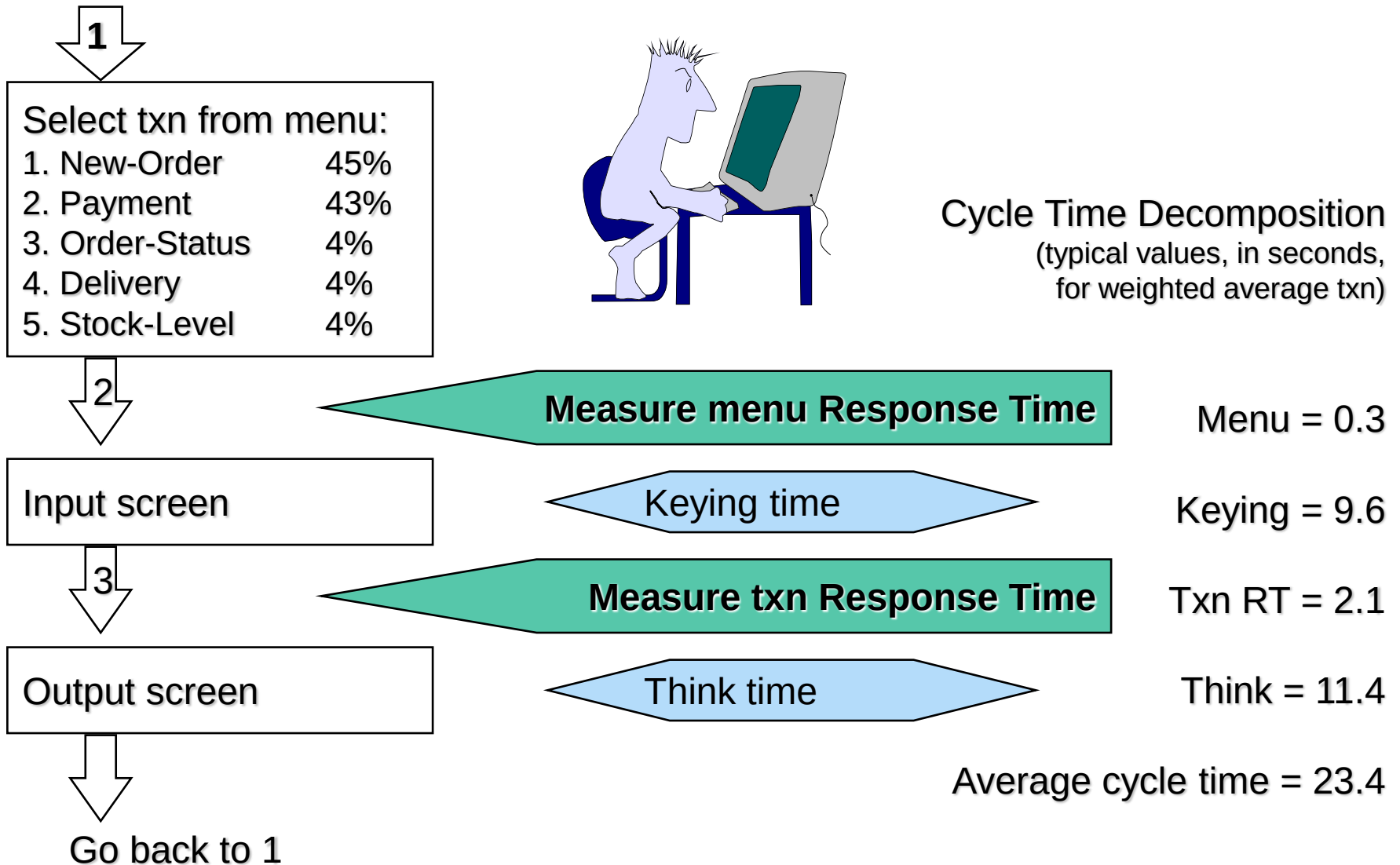
Př.: TPC-C - 5 transakcí

- vlož objednávku od zákazníka (New-order)
- aktualizuj saldo zákazníka dle provedené platby (Payment)
- vyřízení objednávek (Delivery)
- zjisti stav poslední zákazníkovi objednávky (Order-status)
- monitorování skladu (Stock-level)

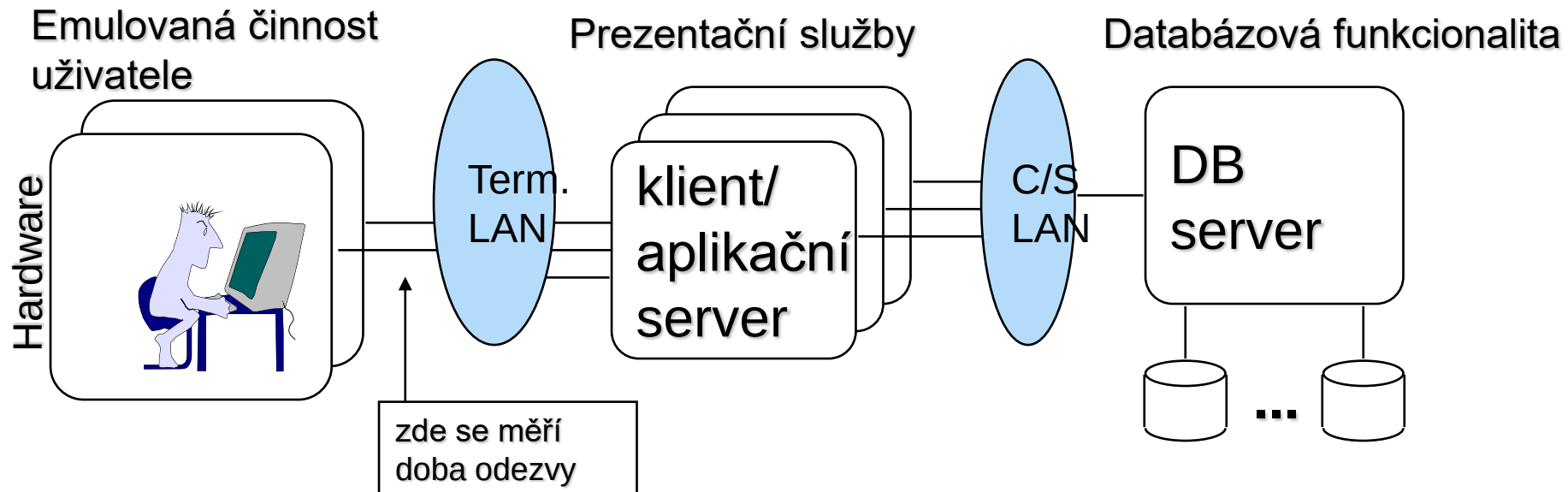
Př.: TPC-C (pokrač.)

- transakce pracují proti databázi obsahující devět tabulek
- transakce provádějí update, insert, delete a také rollback; využívají primární i sekundární klíče
- doba odezvy: 90% transakcí musí mít dobu odezvy ≤ 5 sekund (složitě pak ≤ 20 sekund)





Typická konfigurace pro TPC-C



Software

Např.:
Empower
preVue
LoadRunner

TPC-C aplikace +
transakční monitor
event.
knihovny pro RPC na DB
např., Tuxedo, ODBC

TPC-C aplikace
(uložené procedury) +
databázový server +
transakční monitor
např., SQL Server, Tuxedo

Metody testování

- simulace činnosti uživatele
- testy vstupů a výstupů modulů (black-box)
- testy struktur systému (white-box)
- inspekce (audit)
- porovnání s definovaným standardem (atest)
- sledování a vyhodnocování praktického nasazení (monitor)
- ověření změn (regrese)

Proces testování

Fáze testování:

- definice strategie testování pro vývoj a údržbu programového systému
- tvorba plánu testů
- návrh testů
- tvorba testů
- návrh testovacích cyklů
- provádění a vyhodnocování testů
- změny

1. fáze: Strategie testování I

- obsahuje definici cíle, účelu a pozice testování
- odráží
 - specifická rizika projektu
 - specifické cíle projektu
 - ekonomickou stránku testování a projektu
 - organizaci projektu
 - životní cyklus projektu
- možné koncepty
 - ověření systému v každé fázi životního cyklu
 - určení shody finálního produktu s přihlédnutím k potřebám a požadavkům uživatele
 - ověření chování systému pomocí jeho testovacího provozu při využití testovacích dat

1.fáze: Strategie testování II

- měla by být připravena verze pro každý projekt
- specifikuje
 - systém testování (metody - typy testů a postupy jejich použití)
 - způsob vyhodnocování
 - standardy
 - testovací týmy - role, mapování na organizační strukturu, pravomoci, odpovědnosti

2.fáze: Plánování testů

- **co testovat**
 - jaké jsou klíčové oblasti testované aplikace (DB operace, výkonnost, ...)
 - jaké jsou priority testování (viz analýza rizik)
 - jaké cíle jsou definovány z hlediska jakosti
- **jak testovat**
 - výběr a způsob použití metod testování
 - práce se zjištěnými neshodami (klasifikace závažnosti, vliv na životní cyklus)
 - formalizace
- **kdo bude testovat**
 - definice lidských zdrojů a jejich organizace
 - definice materiálních zdrojů
- **kdy se bude testovat**
 - časový harmonogram návrhu, tvorby, provádění testů

3.fáze: Návrh testů

- definování částí aplikace, které budou samostatně testovány
- určení testu a definování jeho cíle
 - cílem testování je zejména odhalení chyb -> 70% negativních testů
 - testy s komplexnějším záběrem mají větší šanci na odhalení chyb
 - testy by měly být efektivní (účinné, neredundantní)
- určení postupu testu
 - jednotlivé kroky s určením očekávaných výsledků
 - vstupní data (včetně hraničních a nekorektních)
- vychází se z
 - dokumentace programového systému
 - znalostí tvůrců systému
 - eventuálně je nutno použít reverzního inženýrství

4.fáze: Tvorba testů

- vytváření testů dle návrhu
- eventuální korekce navržených kroků testu
- příprava testovacích dat
- specifikace testovacích podmínek (prostředí, konfigurace) a způsobu jejich ustavení
 - opakovatelnost
 - vyloučení vlivu jiného systému
 - dokumentovatelnost

5.fáze: Návrh testovacích cyklů

- testovací cyklus = skupina testů prováděná za určitým účelem
- cyklus se určuje dle specifických cílů a účelu testování
 - v dané fázi vývoje systému (testy modulů, integrační testy, ...)
 - v závislosti na procesu ověřování kvality (základní úroveň, běžná úroveň, nadstavbové testy, speciální testy)
 - v závislosti na předmětu testování (testy modulů, funkcí, subsystémů)

6.fáze: Provádění a vyhodnocování

- provedení jednotlivých kroků testů a dokumentování jejich výsledků
- dokumentace eventuálně zjištěných nesouladů
 - chyby programového systému
 - chyby testu
 - kategorizace chyby (dle škály)
 - návrh řešení

7.fáze: Změny

- cíl: odstranění zjištěných neshod
- postup
 - zjištění příčiny neshody
 - určení způsobu odstranění
 - určení zodpovědného pracovníka
 - dokumentace prováděných změn
 - ohlášení odstranění neshody
 - opakované testování

Je to přijatelné?

AKCEPTAČNÍ TEST

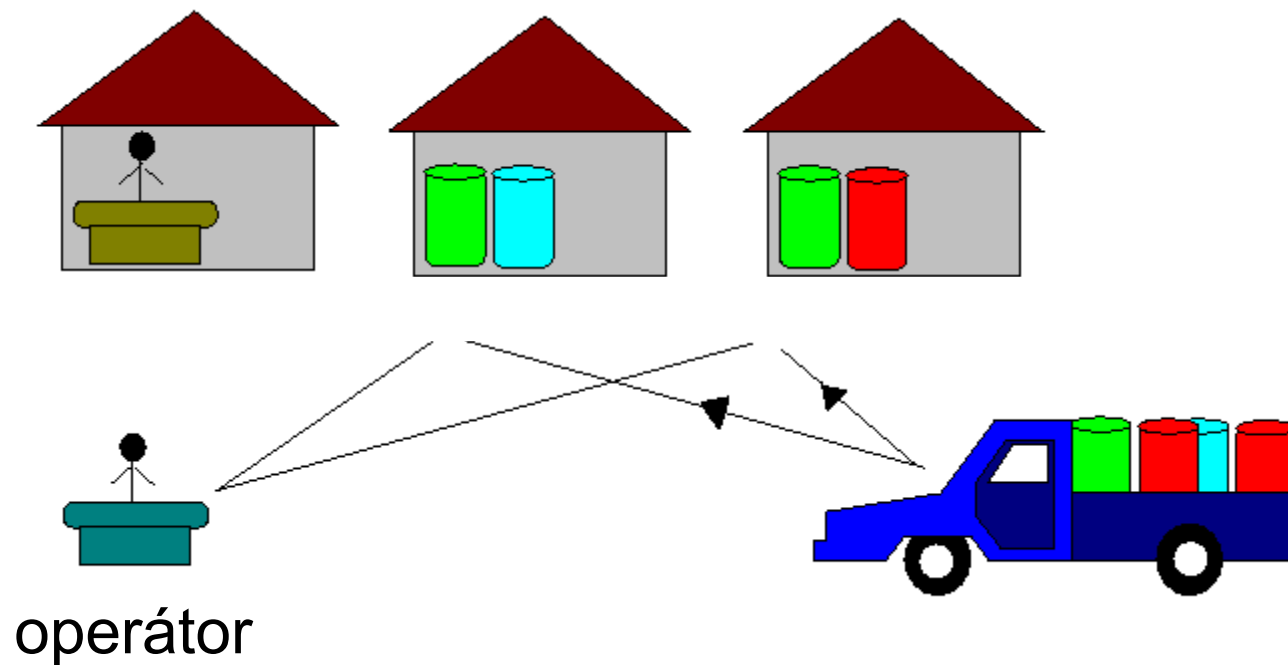
Co to je akceptační test?

- **Akceptační test** představuje podklad pro ověření funkčnosti řešení.
- **Definice akceptačního testu** musí proto obsahovat následující náležitosti:
 1. **podmínky** pro akceptační test
 2. **dokumentaci** pro akceptační test
 3. **definici akcí** pro akceptační test

1. Podmínky pro akceptační test

- Popis prostředí, ve kterém bude akceptační test probíhat. Není-li v akceptačním testu prostředí explicitně stanoveno, musí být možno akceptační test vykonat v rámci standardního prostředí na katedře.
- Popis všech vstupních dat, která budou v akceptačním testu využívána. Patří sem popis všech databází, konfiguračních souborů a jiných testovacích dat, která budou v akceptačním testu využívána.

Příklad projektu ECO-sklad



Podmínky pro akceptační test

Podmínky akceptačního testu pro ECO

- Produkt ECO bude realizován jako formulářová aplikace pro MS-Windows, pracující s daty uloženými v databázi Oracle. Produkt bude vytvořen pomocí nástrojů Designer/2000 a Developer/2000 - Forms. Akceptační test produktu ECO může proto probíhat kdekoli, kde je přístup z MS-Windows k serveru Oracle. Pro provedení akceptačního testu je nutno mít právo přihlásit se jako uživatel do MS-Windows. Dále je nutné mít přístup k nějaké vhodné databázi Oracle jako uživatel, který může instalovat data produktu.

Podmínky pro akceptační test (pokr.)

Podmínky akceptačního testu pro ECO

- Před spuštěním produktu ECO je nutno vytvořit v databázi objekty aplikace a uložit do nich testovací data. Doporučený postup je vytvoření uživatele ECOuser, přidělit mu právo na vytváření objektů a pod tímto uživatelem spustit skript (crECO.sql), který vytvoří potřebné objekty pro ECO a naplní je počátečními testovacími daty. Po absolvování akceptačního testu lze data z databáze odstranit zrušením uživatele ECOuser (s kaskádním odstraněním jeho objektů).

2. Dokumentace akceptačního testu

- Dokumentace potřebná pro vytvoření a instalaci produktu
- Uživatelská příručka
- Definice akceptačního testu
- Protokol o provedení akceptačního testu

Dokumentace akceptačního testu

Dokumentace akceptačního testu pro ECO

- Návod pro instalaci aplikace ECO (zahrnuje popis instalace datové základny a formulářů aplikace)
- Uživatelská příručka produktu ECO
- Definice akceptačního testu ECO
- Protokol o provedení akceptačního testu

Dokumentace akceptačního testu (pokr.)

Dokumentace akceptačního testu pro ECO

Instalace formulářů pro ECO (NT):

- Instalační CD aplikace ECO obsahuje soubor setup.exe
- Pomocí „Start – Nastavení – Ovládací panely – Přidat nebo ubrat programy“ nainstalujete formuláře aplikace ECO.
V domovském adresáři aplikace se vytvoří i soubory potřebné pro instalaci datové základny, včetně testovacích dat a „online“ uživatelské příručky.
- Stejným postupem se formuláře aplikace ECO „odinstalují“.

Dokumentace akceptačního testu (pokr.)

Dokumentace akceptačního testu pro ECO

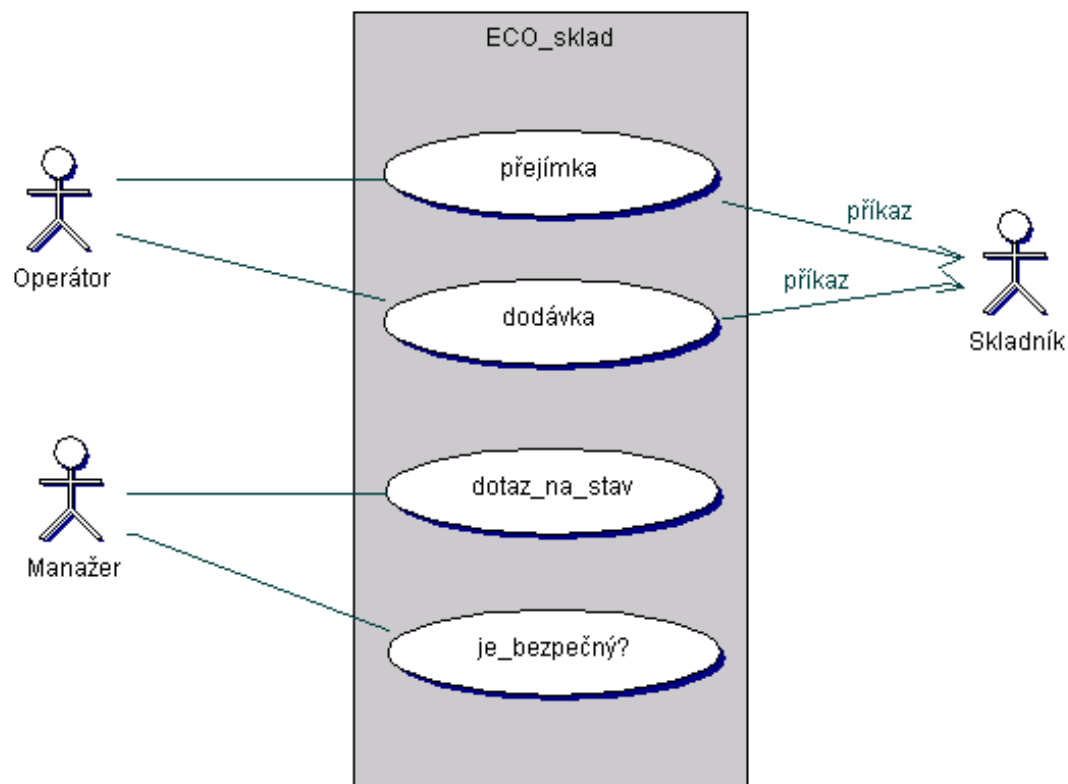
Instalace testovacích dat pro ECO:

- Před spuštěním produktu ECO je nutno vytvořit v databázi objekty aplikace a uložit do nich testovací data. Doporučený postup je vytvoření uživatele ECOuser, přidělit mu právo na vytváření objektů a pod tímto uživatelem spustit skript (crECO.sql), který vytvoří potřebné objekty pro ECO a naplní je počátečními testovacími daty. Po absolvování akceptačního testu lze data z databáze odstranit zrušením uživatele ECOuser (s kaskádním odstraněním jeho objektů).

3. Definice akceptačního testu

- Vychází se z definice aktérů - začíná se kontextem aplikace

Definice akceptačního testu pro ECO



Pro akceptaci ECO je třeba stanovit:

Definice akceptačního testu pro ECO

- Jak se vyzkouší zařazení do role OPERÁTOR a MANAŽER?
- Jak se ověří, že každá role má k dispozici požadovanou sadu služeb?

Akce akceptačního testu

- Popis všech scénářů, které budou tvořit akceptační test. Sada scénářů musí zaručit dostatečné ověření funkčnosti řešení.
- Pro testovací scénáře, pro které je možno stanovit požadovanou reakci systému, je součástí akceptačního testu i popis odpovídajících reakcí.
- Scénáře akceptačního testu musí zahrnovat i základní chybové situace a jejich řešení.
- Vychází se z životního cyklu systému.

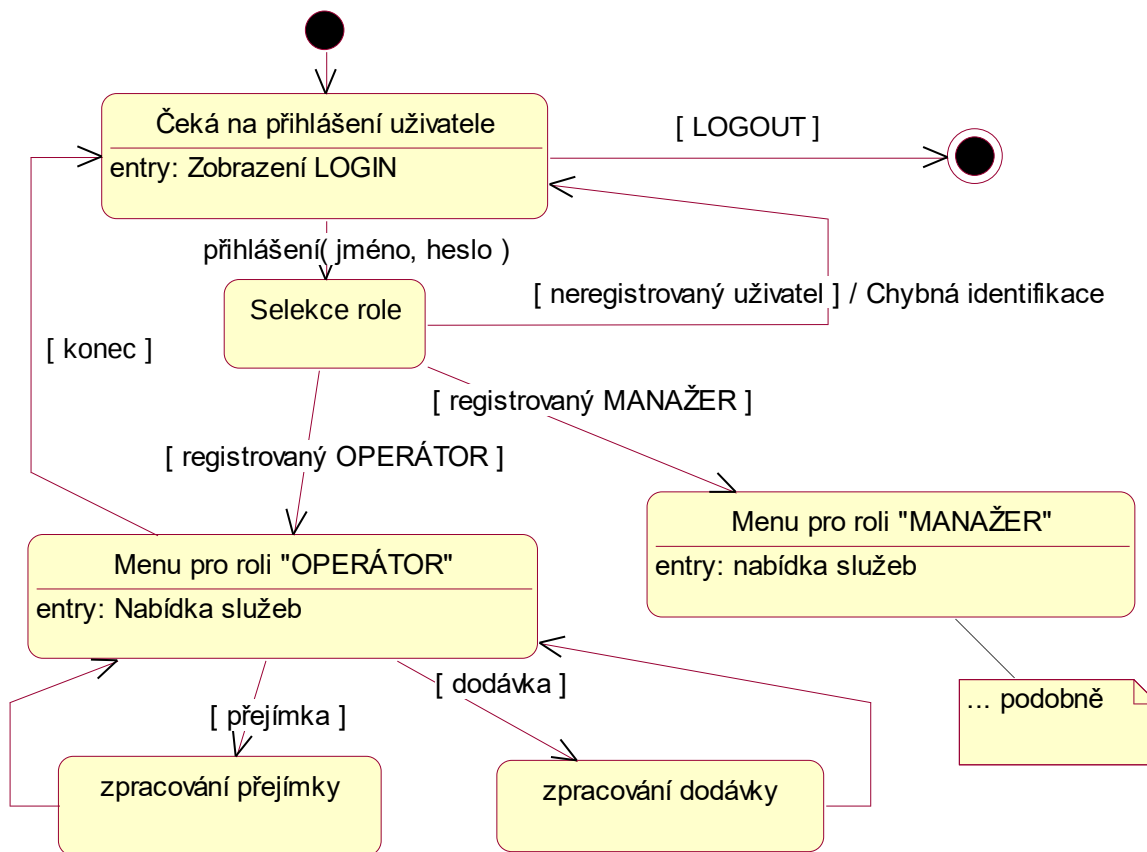
Definice akcí pro akceptační test

Definice akcí akceptačního testu pro ECO

- Akce 1: Instalace a spuštění aplikace ECO
možné reakce:
 - OK
 - nepovedlo se
- Akce 2: Zařazení do rolí OPERÁTOR a MANAŽER
možné reakce:
 - OK
 - nepovedlo se
 - povedlo se, ale nejsou k dispozici služby

Životní cyklus ECO-skladu

Definice akceptačního testu pro ECO



Scénáře pro ECO (viz model jednání)

Definice akceptačního testu pro ECO

- Operátor provádí přejímku
- Operátor vybavuje dodávku
- Manažer se dotazuje na stav skladu
- Manažer zjišťuje, zda je sklad v bezpečném stavu

Pro akceptaci ECO je třeba stanovit:

- Jak se vyzkouší chování při akci dotaz na stav skladu
- Jak se vyzkouší chování při zjišťování, zda je sklad bezpečný
- Jak se vyzkouší chování při akci převímka
- Jak se vyzkouší chování při akci dodávka
- Jak se ověří, že aplikace umí navázat akce

Definice akcí pro akceptační test (pokr.)

Definice akcí akceptačního testu pro ECO

- Akce 3: Dotaz na stav skladu
možné reakce:
 - OK
 - povedlo se - ale chybný výsledek
 - nepovedlo se
- Akce 4: Dotaz na bezpečnost skladu
- Akce 5: Přejímka pro správnou dodávku
- Akce 6: Přejímka pro chybnou dodávku ...

Popis akce

- Identifikace akce
 - Popis: textový popis akce
-
- Co předpokládá
 - Jaké reakce vyvolává (jaké zprávy posílá)
 - Jaká data čte, mění nebo vytváří
 - Co zajišťuje (zaručuje)

Popis pro “Akci 3”

Akce: 3

Popis: Dotaz na stav skladu

- Předpokládá:
- Postup:
 - spuštění funkce dotaz na stav skladu musí vytvořit sestavu zobrazující aktuální stav skladu

Vstupní data pro akci 3:

Definice akceptačního testu pro ECO

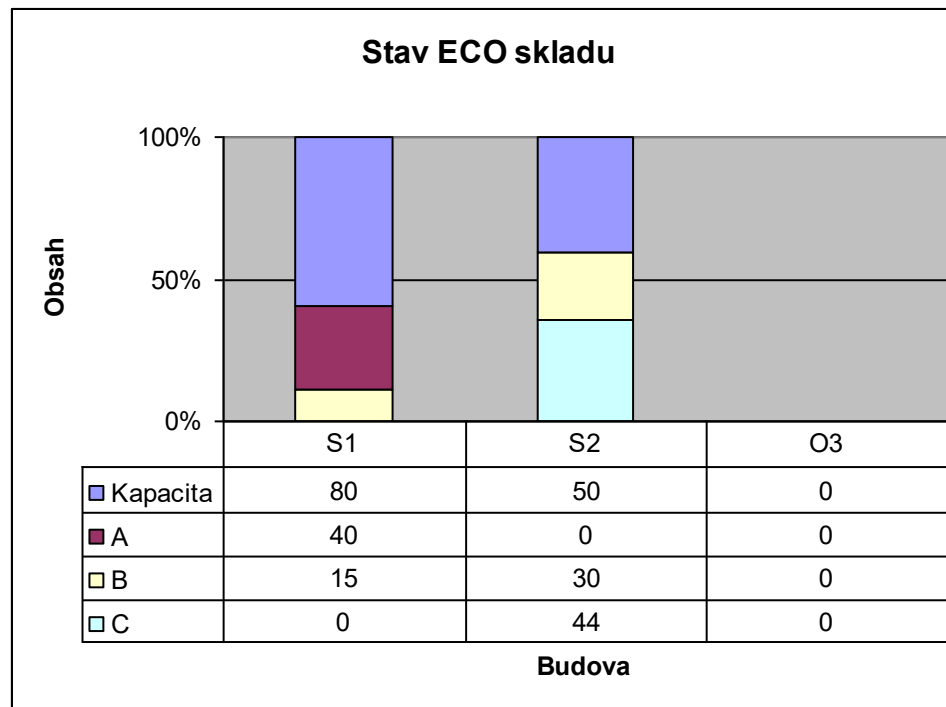
- ECO sklad musí být ve stavu S3 získaném importem souboru ECOS3.dmp příkazem:

```
imp ECOuser/heslo@instance file=ECOS3.dmp
```

Výstupní reakce na akci 3:

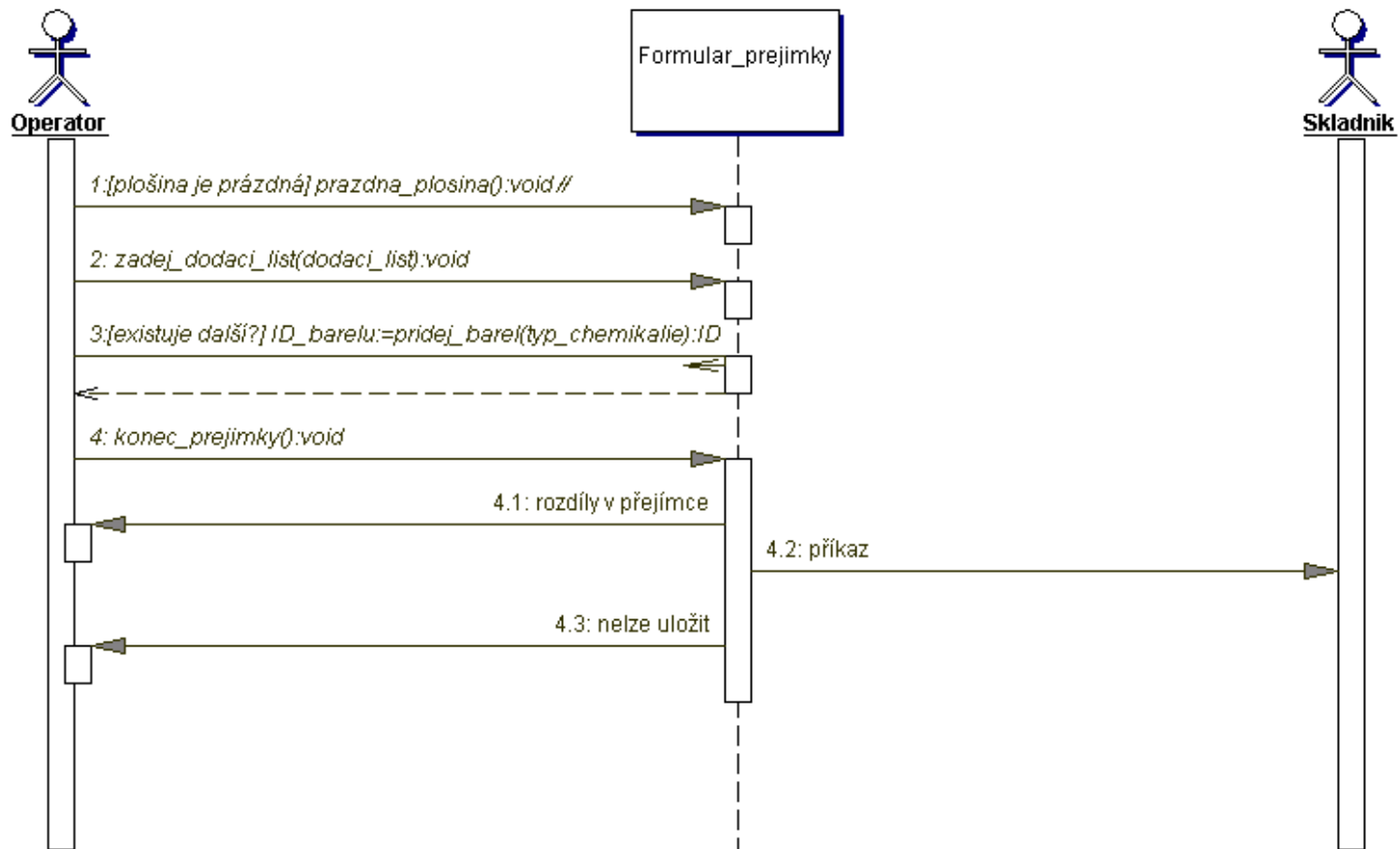
Definice akceptačního testu pro ECO

Pokud je sklad ve stavu S3 měla by akce 3 vytvořit následující sestavu:



Scénář pro “přejímku”

Definice akceptačního testu pro ECO



Životní cyklus “přejímky”

Definice akceptačního testu pro ECO

přejímka = prázdná plošina. dodací list.

(barel k zařazení. #ID barelu)*.

konec přejímky. [#rozdíly v přejímce].

#příkaz pro skladníka . [#nelze uložit]

Musí se vyzkoušet:

- zda produkt nepovolí nesprávné pořadí akcí
- případ správné přejímky, ...

Popis pro “Akci 5”

Akce: 5

Popis: Přejímka pro správnou dodávku

- Předpokládá ECO-sklad v korektním stavu.
- Postup:
 - spuštění funkce přejímka - musí vyvolat formulář pro zadání informací z dodacího listu,
 - zadávají se údaje o barelech - generují se ID barelů (viz Vstupní data 5)
...
- Po ukončení musí být ECO-sklad ve správném stavu (ověří se funkcí “dotaz na stav”) a bezpečný (ověří se funkcí “je bezpečný?”).

Vstupní data pro akci 5:

Definice akceptačního testu pro ECO

- ECO sklad musí být ve stavu S5 získaném importem souboru ECOS5.dmp příkazem

```
imp ECOuser/heslo@instance file=ECOS5.dmp
```

- Dodací list: 5 barelů typu A
4 barely typu B
2 barely typu C
- Postup vykládky:
A, A, B, C, B, C, A, A, A, B, B

Výstupní reakce na akci 5:

Definice akceptačního testu pro ECO

Pokud je sklad ve stavu S5 měla by akce 5 vyvolat následující:

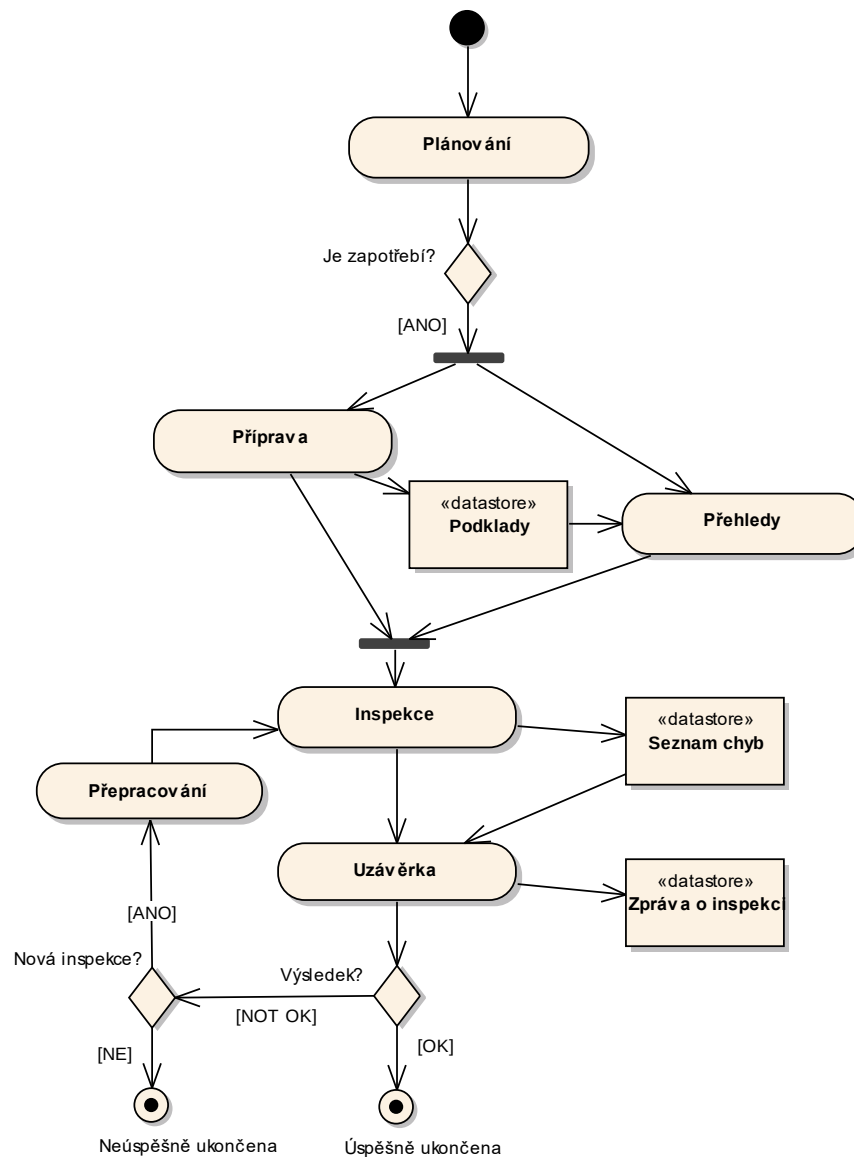
- Nebyly detekovány žádné rozdíly mezi dodacím listem a skutečnou dodávkou
- Nebyly detekovány žádné barely, které nelze do skladu umístit
- Příkaz pro skladníka obsahuje všechny barely dodávky a nikdy neumísťuje barely typu B a C do stejné budovy, celkový počet barelů v budově nepřesáhne kapacitu budovy.

INSPEKCE

Inspekce a revize

- Definice „inspekce“
- Způsob organizace inspekce
- Participanti jednotlivých fází inspekce
- Výběr moderátora

act Inspekce



Inspekční tým - 3 až 7 lidí

- Role:
 - **autor** - osoba, která je autorem produktu a je zodpovědná za změny řešící nalezené problémy
 - **moderátor** - osoba, která zajišťuje průběh inspekce podle připraveného plánu
 - **čtenář** - osoba, která překládá produkt k inspekci
 - **zapisovatel** - osoba, která zaznamenává indikované chyby a spolupracuje s moderátorem na přípravě zprávy o inspekci
 - **inspektor** - osoba, která se v předkládaném produktu snaží nalézt chyby

Poznámky k inspekci:

- Minimální inspekční tým má tři osoby - autor, čtenář a moderátor/zapisovatel (všichni jsou současně inspektory).
- Autor musí být vždy přítomen a nesmí zastávat žádnou jinou roli (s výjimkou inspektora).
- Inspekční tým by měl být přiměřený - nejvíce asi 7 osob, pokud je to vhodné.
- Inspekce není určena pro manažery.

Metody pro výběr moderátora

- Moderátor je přidělen autorům již při vytváření plánu projektu.
- Moderátor je vybrán koordinátorem inspekci.
- Autor si vybírá moderátora ze seznamu pověřených moderátorů.
- Autor si vybírá moderátora sám.

Plánování inspekce

- Účel: organizace inspekčního procesu
- Úlohy:
 - Vymezení nebo potvrzení vstupních kritérií (pro přijetí produktu k inspekci)
 - Ustavení plánu (inspekce by rozhodně neměla trvat déle než 2-3 hodiny)
 - Výběr participantů
 - Rozhodnutí o přehledech (přehledy slouží pro seznámení s produktem)
 - Příprava podkladů pro inspekční setkání (typ inspekce, předmět inspekce, doba a místo konání přehledů, doba a místo konání inspekce, odhad času, požadovaná příprava)
 - Distribuce materiálů participantům
- Role: moderátor, autor

Přehledy a příprava na inspekci

- **Přehledy (overview)**
 - Účel: seznámení s produktem nebo předmětem (volitelné)
 - Úlohy: Prezentace
 - Role: moderátor, autor, inspektoři, ostatní
- **Příprava**
 - Účel: porozumění materiálům, potenciální identifikace chyb
 - Úlohy: Studium materiálů
 - Role: všichni inspektoři

Inspekční setkání

- Účel: ověření produktu
- Úlohy:
 - Otevření inspekčního setkání (seznámení s obsahem, postupem a kritérii)
 - Ověření připravenosti participantů (bez přípravy nemá inspekce cenu)
 - Čtení produktu (čtenář prezentuje produkt), indikace a záznam chyb
 - Zkontrolování seznamu chyb (zapisovatel prezentuje zaznamenané chyby)
 - Vyhodnocení a závěry inspekce (A - akceptovat bez další inspekce, B - další akceptace ponechána na moderátorovi, C - vyžaduje novou inspekci)
- Role: moderátor, autor, čtenář, zapisovatel, inspektoři

Uzávěrka inspekce

- **Přepracování**
 - Účel: splnění výstupních kritérií
 - Úlohy: vyřešení všech chyb
 - Role: autor
- **Uzávěrka**
 - Účel: Potvrzení inspekce
 - Úlohy: Ověření úprav produktu
 - Zpráva o výsledku inspekce
 - Role: moderátor, autor, (inspektoři)

The End